

$$R \cup S : \{x / x \in R \vee x \in S\} \equiv \begin{array}{|c|} \hline A \\ \hline 2 \\ \hline 3 \\ \hline 4 \\ \hline \end{array}$$

$$R - S : \{x / x \in R \wedge x \notin S\} \equiv \begin{array}{|c|} \hline A \\ \hline 3 \\ \hline \end{array}$$

$$R \cap S : \{x / x \in R \wedge x \in S\} \equiv \begin{array}{|c|} \hline A \\ \hline 2 \\ \hline \end{array}$$

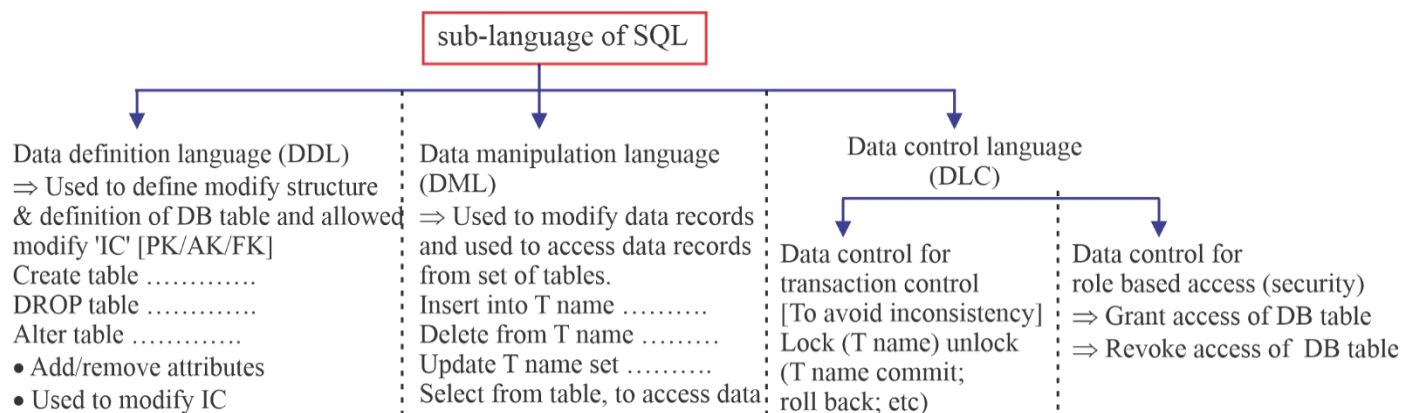
3.7 Cardinality Table

Assume relation R with n Distinct tuples and relation S with m distinct tuples:

RA Expression	Cardinality
$R \times S$	$n * m$ tuples
$R \bowtie S$	{0 to $n * m$ tuples}
$R = \bowtie S$	{n to $n * m$ tuples}
$R \bowtie = S$	{m to $n * m$ tuples}
$R = \bowtie = S$	{max (n,m) to $n * m$ tuples}
$R \cup S$	{max(n,m) to $n + m$ }
$R \cap S$	{0 to min (m, n)}
$R - S$	{0 to n tuples}
R / S	{0 to $[n/m]$ tuples}

3.8 SQL

3.8.1 [Structure Query Lang]



3.8.2 Relational Query and SQL query

- SQL : SELECT DISTINCT $A_1, A_2, \dots, A_n$

FROM $R_1, R_2, \dots, R_n$

WHERE P ;

Equivalent RA : $\pi_{A_1, A_2, \dots, A_n} (\sigma_P (R_1 \times R_2 \times \dots \times R_n))$

- SELECT $\equiv$ projection of RA (π)
- FROM $\equiv$ Cross-product ($\times$)
- WHERE $\equiv$ Selection operator of RA (σ)

3.8.3 Basic SQL Clauses

SELECT DISTINCT $A_1, A_2, \dots, A_n$

FROM $R_1, R_2, \dots, R_n$

WHERE condition

GROUP BY (attributes)

HAVING condition

ORDER BY (attributes) (DESC)

3.8.4 Execution Flow

FROM $\Rightarrow$ cross-product of relations

↓

WHERE $\Rightarrow$ Selection operator (σ) to apply condition for each record

↓

GROUP BY

↓

HAVING

↓

SELECT

↓

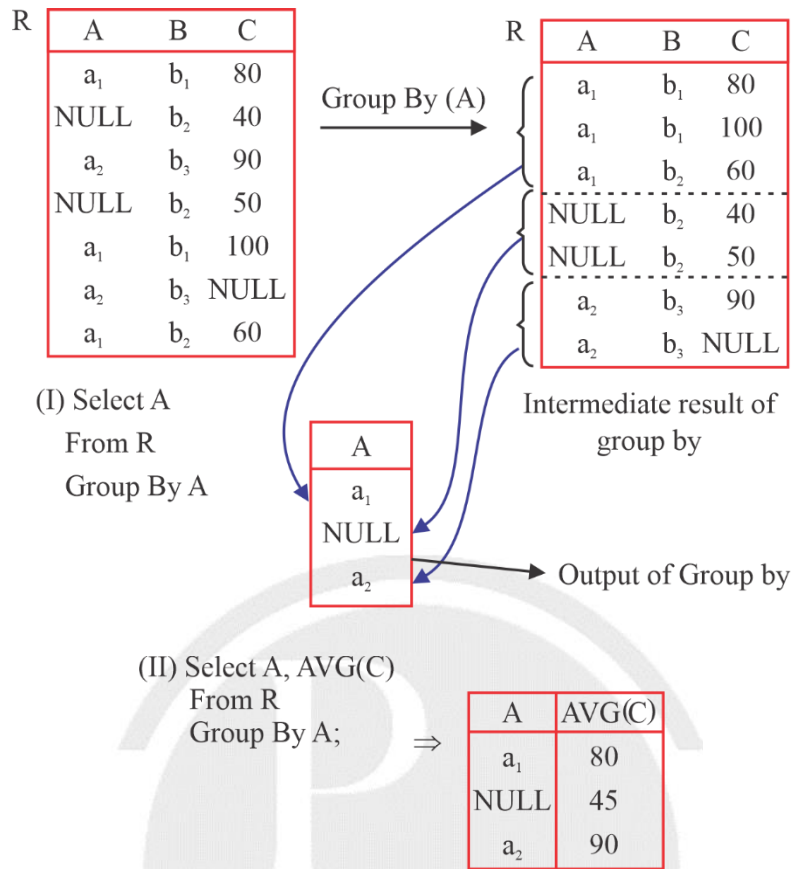
DISTINCT

↓

ORDER BY

3.8.5 GROUP BY:

- It is used to group records data a based on specific attribute.
- It GROUP BY clause used then
 - (a) Every attribute of GROUP BY clause must be selected in SELECT clause.
 - (b) Not allowed to select any other attribute in SELECT clause.
 Allowed to select aggregate function along with GROUP BY attribute in SELECT Clause.



3.8.6 HAVING Clause

- HAVING clause must be followed by GROUP BY clause.
- HAVING clause used to select groups those are satisfied having clause condition.
- HAVING clause condition must be over aggregation function such as some (), Every () etc. but not allowed direct attribute comparison

Example:

- ```
SELECT A
FROM R
GROUP BY A
HAVING AVG (c) > 60;
```
- ```
SELECT A, AVG (c)
FROM R
GROUP BY A
HAVING some (c) > 50;
```
- ```
SELECT A
FROM R
GROUP BY A
HAVING C > 60;
```

} Error



⇓  
Not allowed

**Note:**

WHERE clause condition tested for each record but HAVING clause condition tested for each GROUP.

### 3.8.7 WITH Clause

- WITH clause is used to create sub-query result that can be re-used many times in query processing.

**Example:**

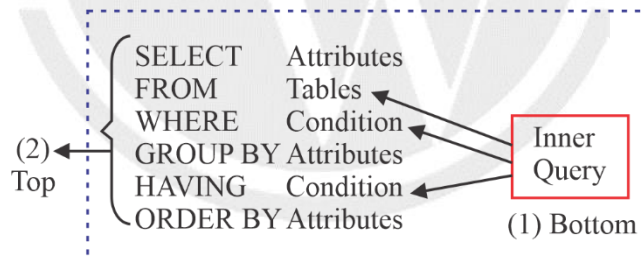
```
WITH Temp (A1, A2) as
(SELECT T1 . A1, T2 . A2
FROM Emp T1, Emp T2
WHERE T1 . A1 < T2 . A1)
```

**Note:**

Every Query which is written by using HAVING clause can be re-written by using WHERE clause.

### 3.8.8 Nested Query Without Co-relations

- Inner query independent of outer query.



- Execution flow must be Bottom to Top mean first bottom (Inner query) evaluated then outer query executed.

### 3.8.9 Co-related Nested Query

- In Nested Co-related query inner query uses attributes from outer query tables.
- In Co-related Nested query inner query allowed in WHERE, HAVING clause of outer query.

**Example:**

```
SELECT A
FROM R
WHERE (SELECT count(*)
FROM S
WHERE S.B < R.A) < 5;
```

### Important point 1

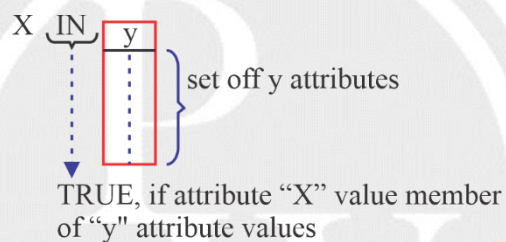
- If co-relation in WHERE clause then inner query re-computes for each record of outer query From clause.
- If correlation in HAVING clause then inner query re-computes for each group of outer query.

## 3.9 Function used for nested Queries

- |                      |                                              |
|----------------------|----------------------------------------------|
| 1. IN/ NOT IN        | } Best suitable for<br>non- co-related query |
| 2. ANY               |                                              |
| 3. ALL               |                                              |
| 4. EXISTS/NOT EXISTS | } Best suitable for<br>co-related query      |

### 1. IN Function

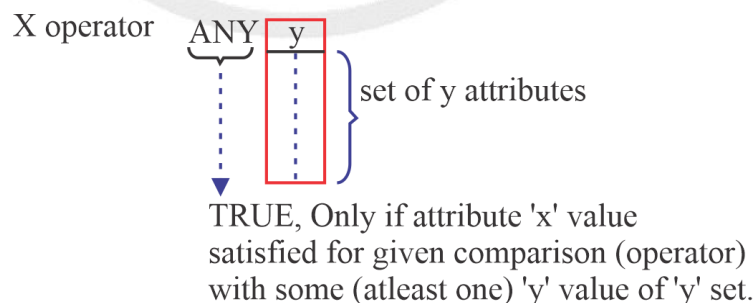
It is used for membership testing.



#### Note:

Queries of "equi Join" can implement by IN function  $\equiv (R \bowtie S)$

### 2. ANY Function (operator "<, ≤, >, ≥, =, <>")



### 3. ALL Function

X operator  $\underbrace{\text{ALL}}_{\downarrow} \underbrace{\begin{array}{|c|} \hline y \\ \hline \end{array}}_{\text{set of 'y' attributes}}$

TRUE, Only if attribute 'x' value satisfied for given comparison (operator) with every (all) 'y' value of 'y' set.

**Note:**

ANY function can be used as queries of conditional join query.

**Example:**

$$\pi_A \left( R \bowtie_{R.A > S.B} S \right) \equiv \text{SELECT A FROM R WHERE R.A > ANY (SELECT B FROM S)}$$

$$\pi_A(R) - \pi_A \left( R \bowtie_{R.A \leq S.B} S \right) \equiv \text{SELECT A FROM R WHERE R.A > ALL (SELECT B FROM S)}$$

**Important point 2**

- IN function equal to '= ANY' function

$$X \text{ IN } \begin{array}{|c|} \hline Y \\ \hline \end{array} \equiv X = \text{ANY} \begin{array}{|c|} \hline Y \\ \hline \end{array}$$

- IN function not equal to '= ANY' if more than one attribute used for IN comparison.

$$(A,B) \text{ IN } \begin{array}{|c|c|} \hline C & D \\ \hline \end{array} \neq (A,B) = \underbrace{\text{ANY}}_{\text{Not allowed}} \begin{array}{|c|c|} \hline C & D \\ \hline \end{array}$$

- NOT IN function equal to "<> ALL".

$$X \text{ NOT IN } \begin{array}{|c|} \hline Y \\ \hline \end{array} \equiv X <> \text{ALL} \begin{array}{|c|} \hline Y \\ \hline \end{array}$$

**4. EXISTS clause**

- It is used to test result of inner query is empty or not empty.
-



EXISTS (Query)

TRUE, if result of inner query is non-empty that is atleast one record in result of inner query.

**Example:**

```
SELECT R.A
FROM R
WHERE EXISTS (SELECT *
 FROM S
 WHERE R.A > S.B);
```

Atleast one record of S relation satisfy R.A > S.B

**Note:**

EXISTS function can use as conditional join queries with join condition in inner query WHERE clause.

Emp (eid sal dno. Gen) Retrieve eids of femal's whose salary more than (some) male emp.

$$RA: \pi_{eid} \sigma_{gen = Female}(Emp) \bowtie \rho_{I,S,D,G} (\sigma_{gen=male}(emp))$$

$$Sal > S$$

SQL

[JOIN Query]

```
SELECT DISTINCT T1 . eid
FROM EMP T1, Emp T2
WHERE
T1.gen = Female and
T2.gen = male and
T1.sal > T2 . sal
```

SQL

[Mested Query]

```
SELECT Eid
FROM Emp
WHERE gen = Female
and sal > ANY (SELECT sal
 FROM Emp
 WHERE gen = male);
```

SQL

[Co – related Nested Query]



```
SELECT Eid
FROM Emp T1
WHERE T1.gen = Female and
EXISTS (SELECT *
 FROM Emp T2
 WHERE T2.gen = male and T1. Sal > T2 . sal);
```

